UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SOFTWARE DESIGN AND ARCHITECTURE (SDA)

Chapter 5: Principles of Detail Design

Innovating Solutions

UTMSPACE ESTD 1993 (931427-K)

01 **DETAIL DESIGN**

Overview and Principle of Detail Design

02 **KEY TASKS DETAIL DESIGN**

Understanding, Creating, Evaluating, Documenting and Monitoring

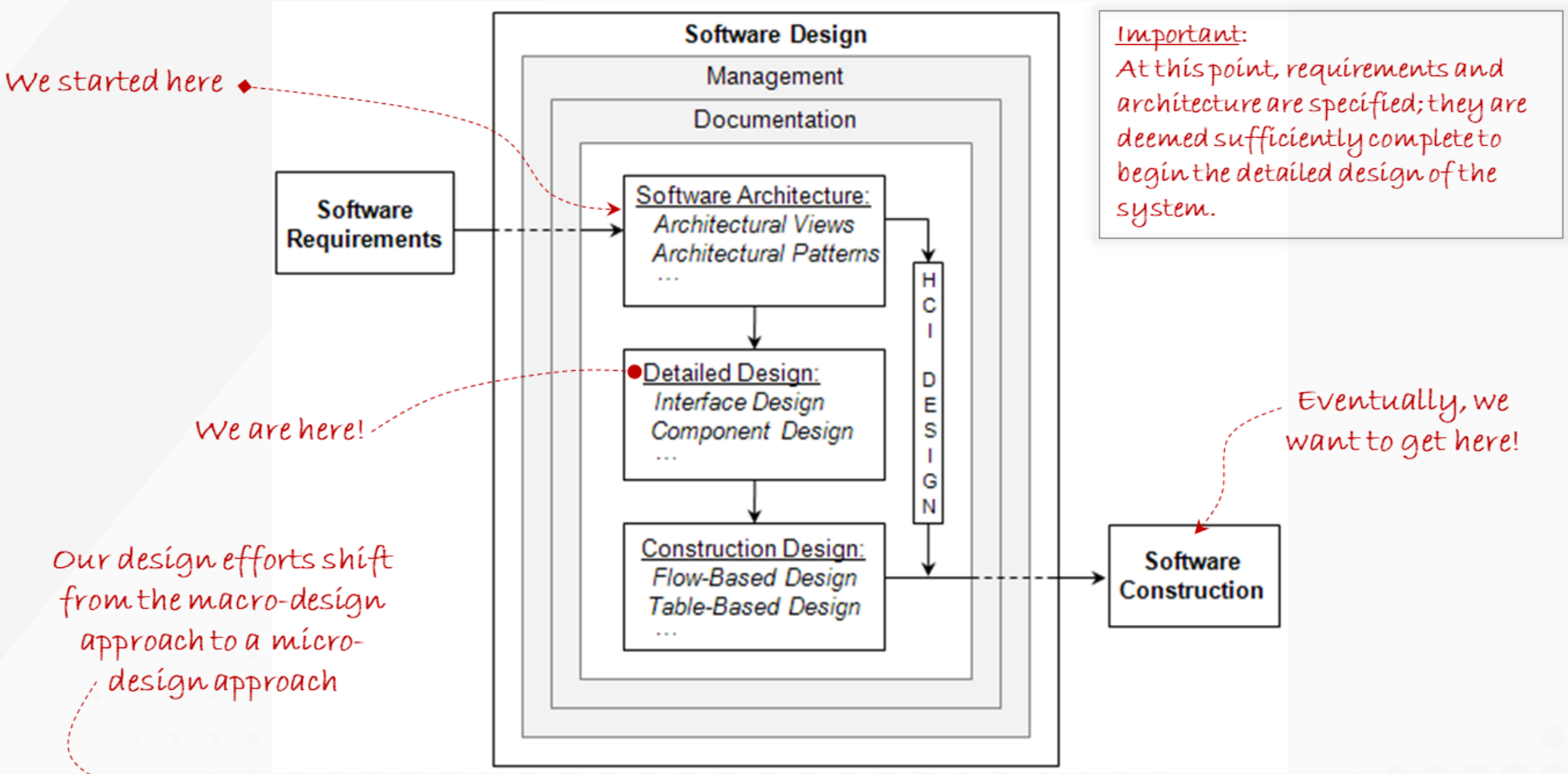
03 **INTERNAL & COMPONENT DESIGN**

OO Approach

04 **DETAIL DESIGN PRINCIPLE**

OCP, LSP and ISP.

THE DETAIL DESIGN



ARCHITECTURAL VS DETAIL DESIGN

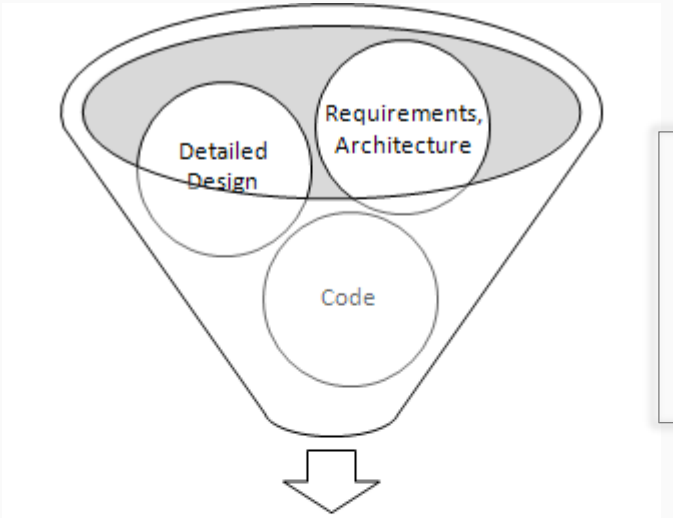
Aspect	Architectural Design (Style)	Detailed Design (Pattern)
Scope	Focuses on high-level system structures	Deals with specifics of individual components
Abstraction Level	Conceptual	Concrete
Granularity	Large-scale components and interactions	Finer-grained details like algorithms, data structures
Decision Making	Strategic decisions about system organization	Tactical decisions about implementation details
Focus	Performance, scalability, flexibility	Functionality and efficiency of modules
Documentation	System architecture, interfaces, interactions	Detailed design decisions, data structures, algorithms



WHAT IT LOOK LIKES

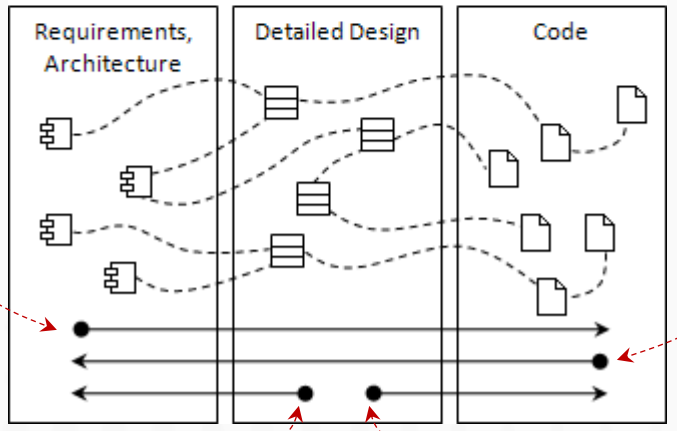


Designer's Mental Model During Detailed Design!



Important:
During detailed design, the use of industry-grade development tools are essential for modeling, code generation, compiling generated code, reverse engineering, software configuration management, etc.

If given requirements and architecture, detailed designers must move the project forward all the way to code



If given code, detailed designers must be able to reverse engineer the code to produce detailed and architectural designs.

When starting at detailed design, designers must be able to produce both code and architectural designs

THE KEY TASK IN DETAIL DESIGN

01. UNDERSTANDING REQUIREMENTS

Focus on requirements allocated to their specific components

02. CREATING THE DESIGN

Detailed design consist of both structural and behavioral designs, interfaces and GUI

03. EVALUATING THE DESIGN

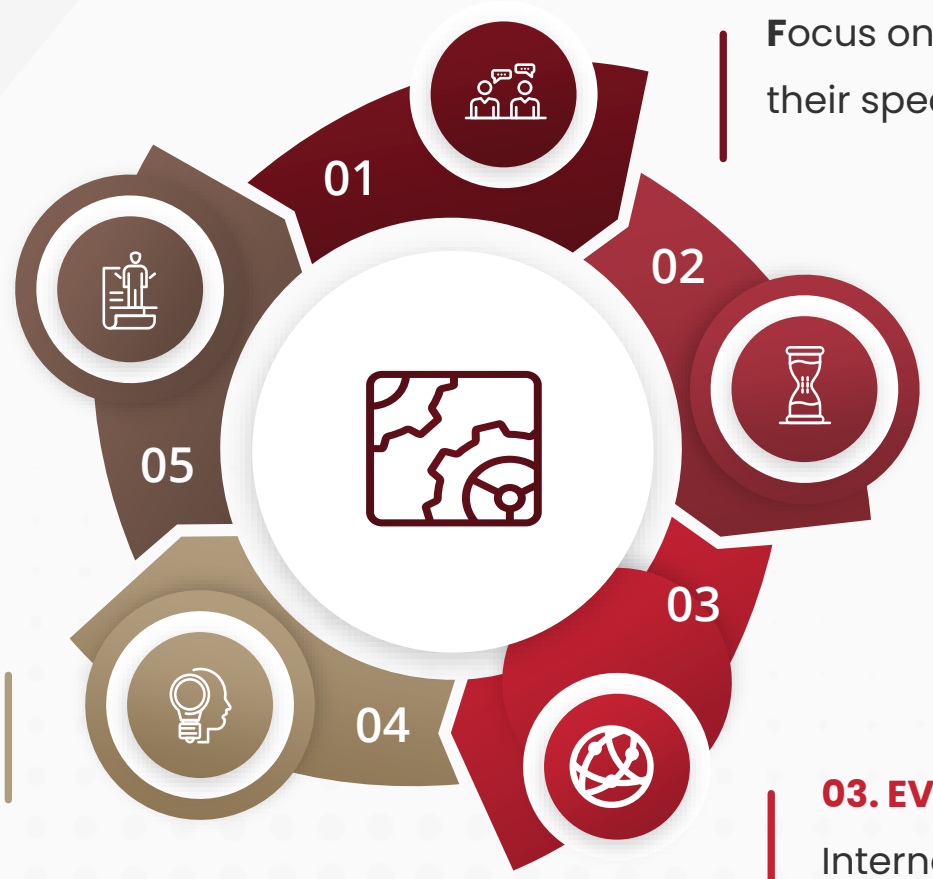
Internal Technical Review.

04.DOCUMENTING

SOFTWARE DESIGN DOCUMENT (SDD)

05.MANAGING & MONITORING

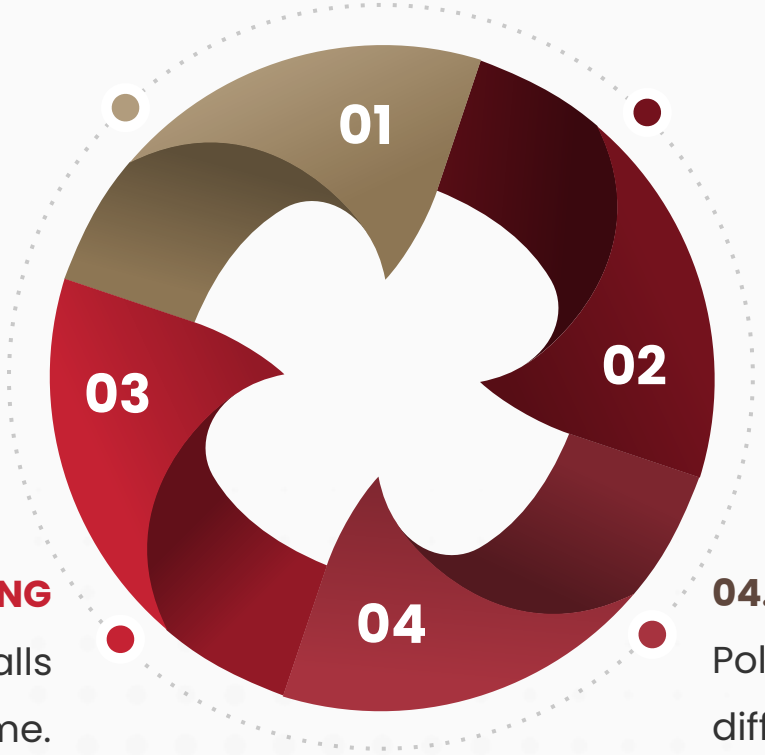
Architectural to Detail Design to Code.



INTERNAL & EXTERNAL COMPONENT DESIGN

01. CLASSES AND OBJECTS

Objects are instances of classes, representing real-world entities with specific characteristics and behaviors.



02. INTERFACES, TYPES, AND SUBTYPES

Interfaces define a contract for classes to implement specific methods. Subtypes inherit properties and behaviors from parent types or interfaces.

03. DYNAMIC BINDING

Dynamic binding allows method calls to be resolved at runtime.

04. POLYMORPHISM

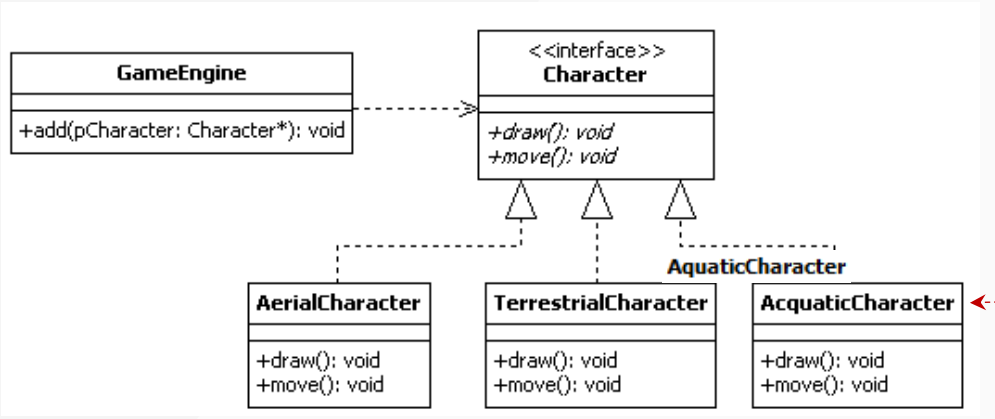
Polymorphism allows objects of different classes to be treated as objects of a common superclass through inheritance

DETAIL DESIGN PRINCIPLE

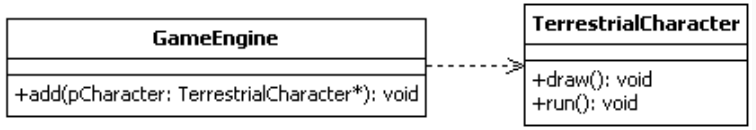
Principle	Open-Closed Principle (OCP)	Liskov Substitution Principle (LSP)
Definition	Software entities should be open for extension but closed for modification.	Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.
Focus	Emphasizes designing modules that can be easily extended with new functionality.	Focuses on ensuring that inheritance hierarchies are designed in a way that does not break the behavior of the superclass when substituting objects of its subclasses.
Objective	Promotes maintainability and extensibility by minimizing the need for modifying existing code.	Ensures that clients can use subclasses interchangeably with their parent classes, allowing for flexibility and modularity in the design.
Implementation	Achieved through techniques like abstraction, encapsulation, and inheritance.	Implemented by adhering to guidelines such as not weakening preconditions, not strengthening postconditions, and maintaining behavioral consistency in subclasses.
Example	Using abstract classes and interfaces to define contracts, allowing new implementations to be added without modifying existing code.	Ensuring that derived classes do not violate the expected behavior of the base class, such as a Square inheriting from a Rectangle without breaking the Rectangle's behavior regarding width and height relationships.

OPEN-CLOSED PRINCIPLE (OCP)

New redesign! Adheres to OCP!



Old design! Violates OCP!



New Aquatic Character added by extension and not by modifying existing working code!

```
// The game engine responsible for managing the game.
class GameEngine {

public:
  // Add the character to the screen.
  void add(Character* pCharacter) {

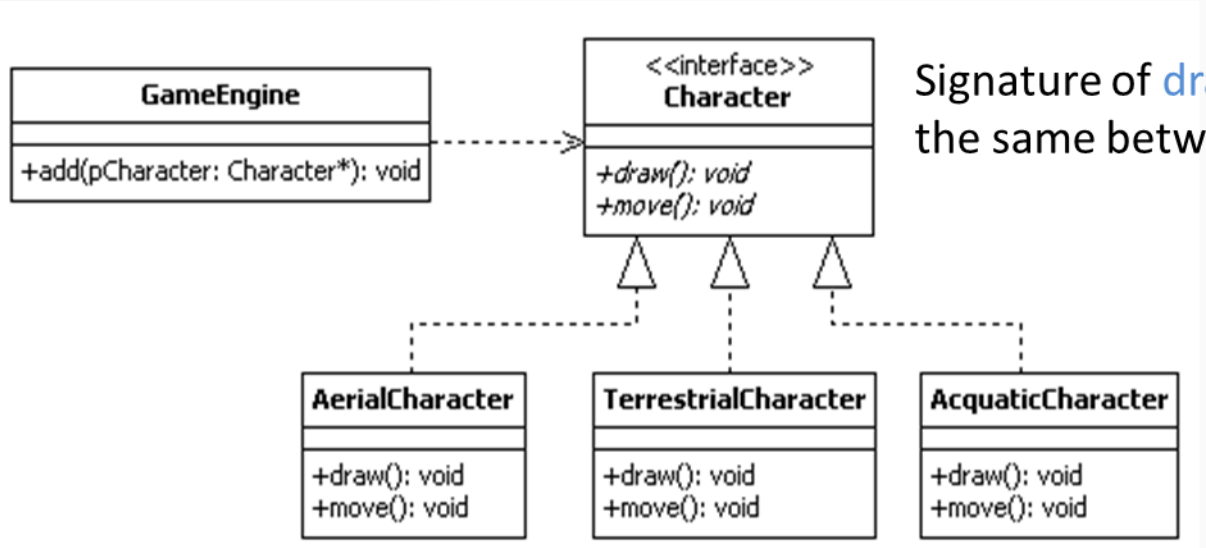
    // Display the character.
    pCharacter->draw();

    // Activate the character... make it move!
    pCharacter->move();

  } // end add function.
};
```

With this design, GameEngine can draw and activate current and future Characters in the game without modification!

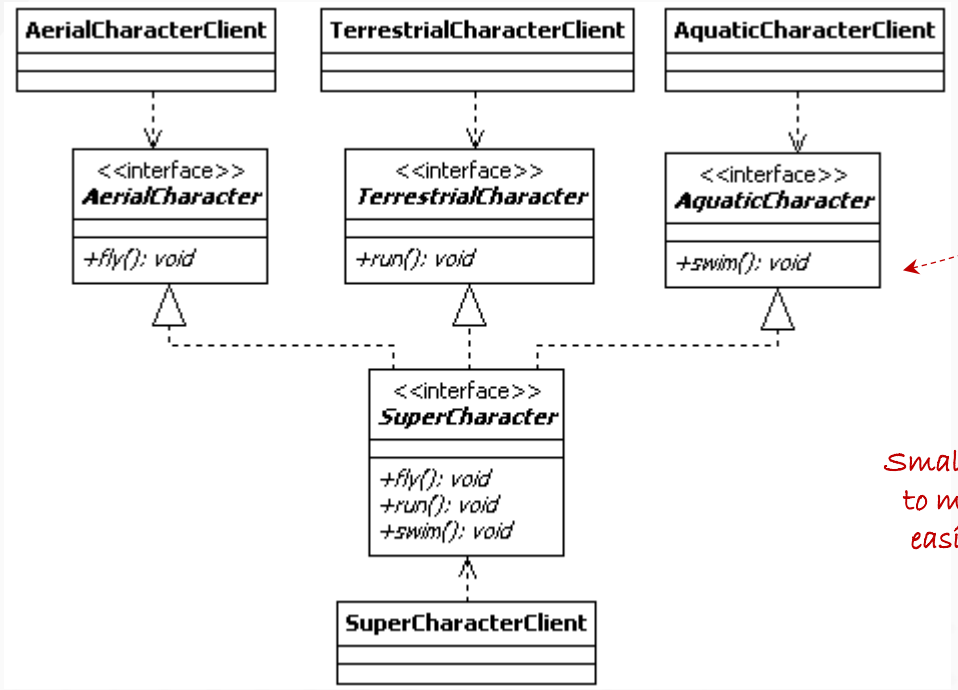
LISKOV SUBSTITUTION PRINCIPLE (LSP)

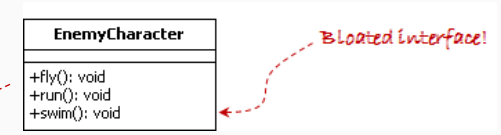


Signature of draw() and move() method is the same between supertype and subtype

INTERFACE SEGREGATION PRINCIPLE (ISP)

The interface segregation principle (ISP) states that "clients should not be forced to depend on methods that they do not use“





The ISP implies that many client-specific interfaces are better than one general purpose interface

Smaller, specific interfaces are easier to maintain and reuse and lead to easier adherence to OCP and LSP



Thank you for your participation. I am very grateful to have the opportunity into this sharing session. If there any enquiries, feels free to contact me.

+013 509 9215

khatibsyarbini@utm.my

THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

utm.my univteknologimalaysia utmofficial